Introduction SQL Analytics on Lakehouse Architecture

Instructor: Doug Bateman

databricks

About Your Instructor

Doug Bateman



- Principal Data Engineering Instructor at Databricks
- Joined Databricks in 2016
- 20+ Years of Industry Experience

About Your Instructor (Personal)

Doug Bateman



- Two children
  - 2 and 5 years old
- For fun:
  - Sailing
  - Rock Climbing
  - Snowboarding (badly)
  - Chess (badly)

Course goals

1. Describe key features of a data Lakehouse
2. Explain how Delta Lake enables a Lakehouse architecture
3. Define key features available in the Databricks SQL Analytics user interface.

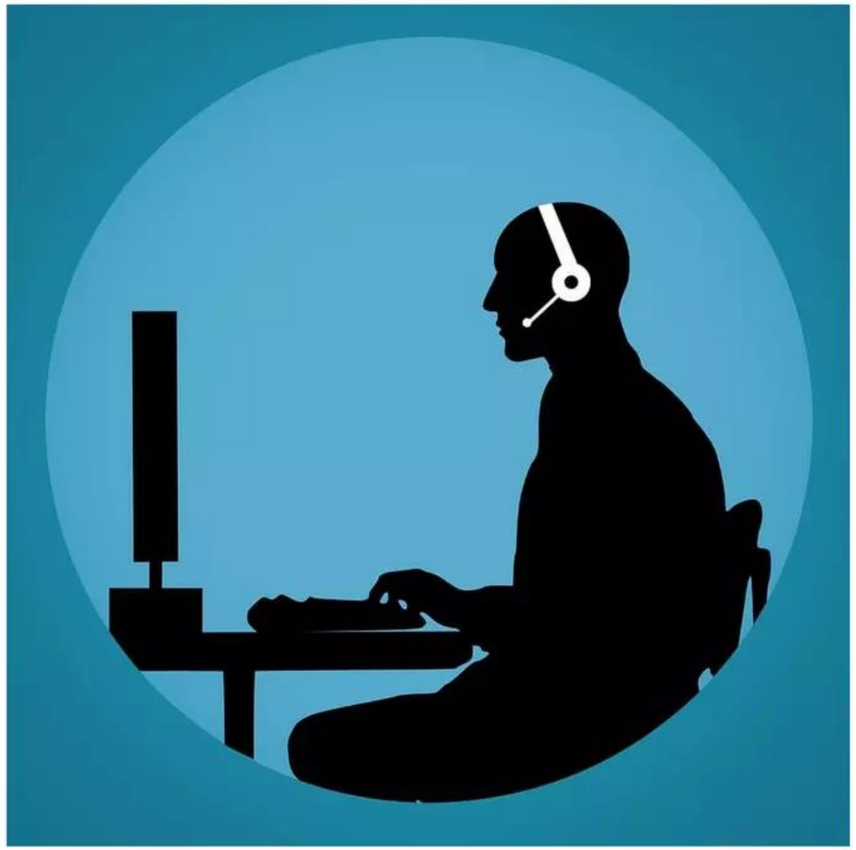
Course Agenda

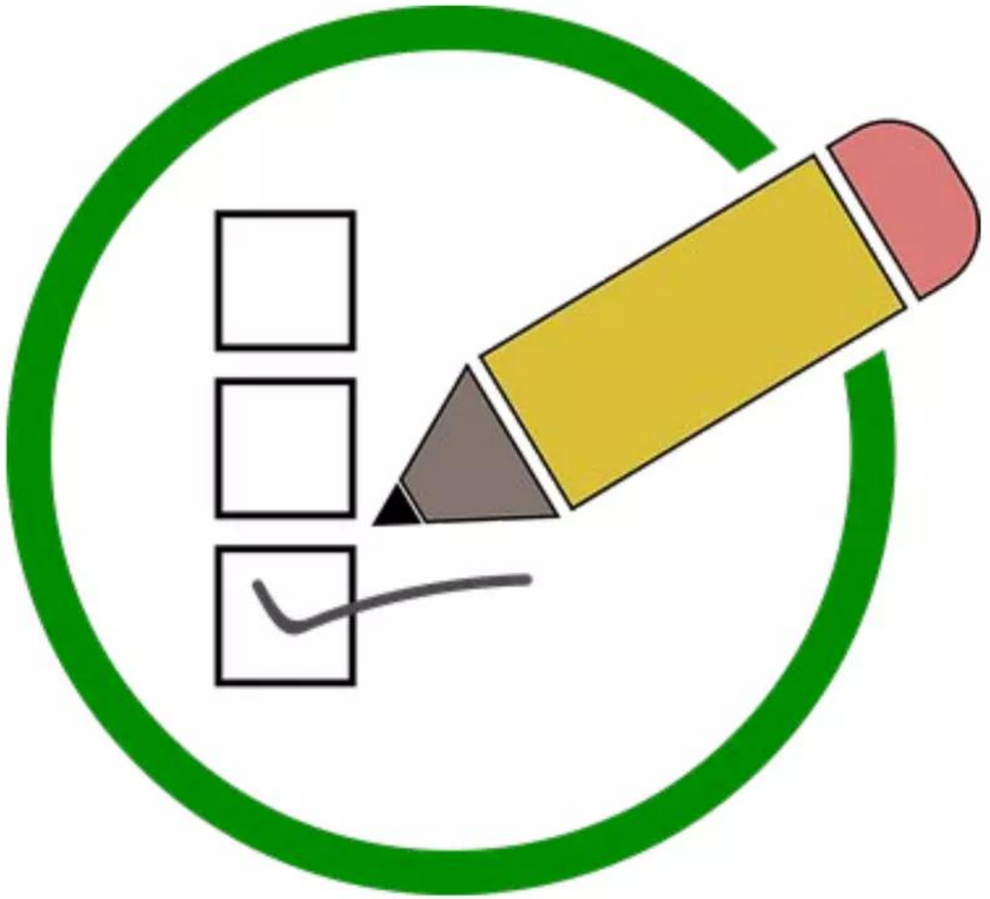
Activity
Course welcome
Introduction to Lakehouse Architecture
Delta Lake
Databricks SQL Analytics Intro
Databricks SQL Analytics Demo
Wrap up and Q & A

Access the Slides

https://tinyurl.com/lakehouse-webinar

About You (Polls)





Introduction to Lakehouse Architecture



Data Driven Decisions



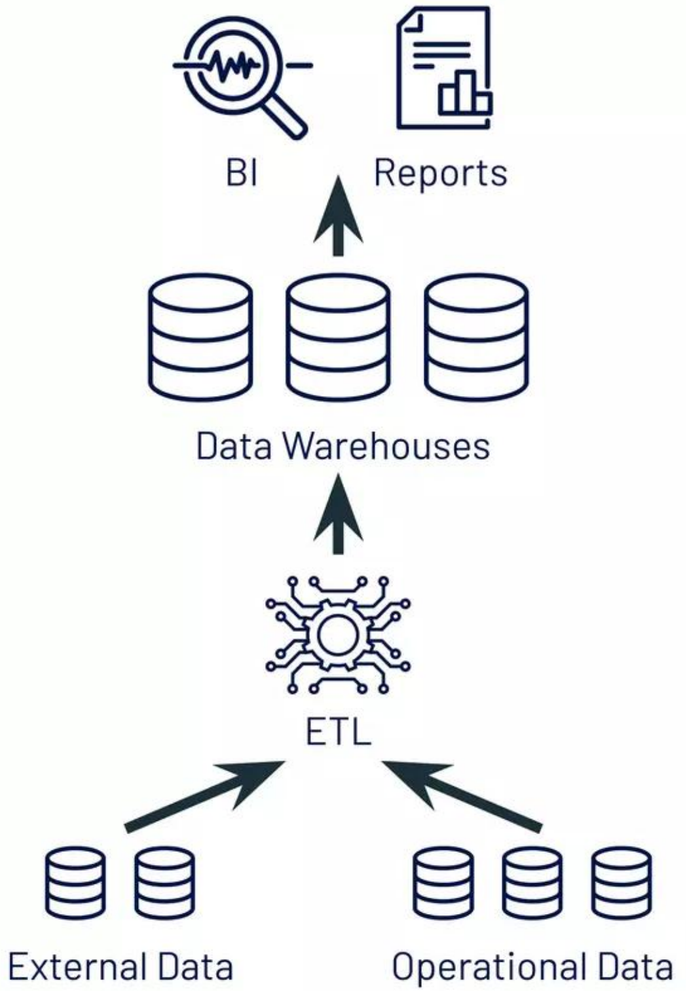




Data Warehouses
were purpose-built
for BI and reporting,
however...

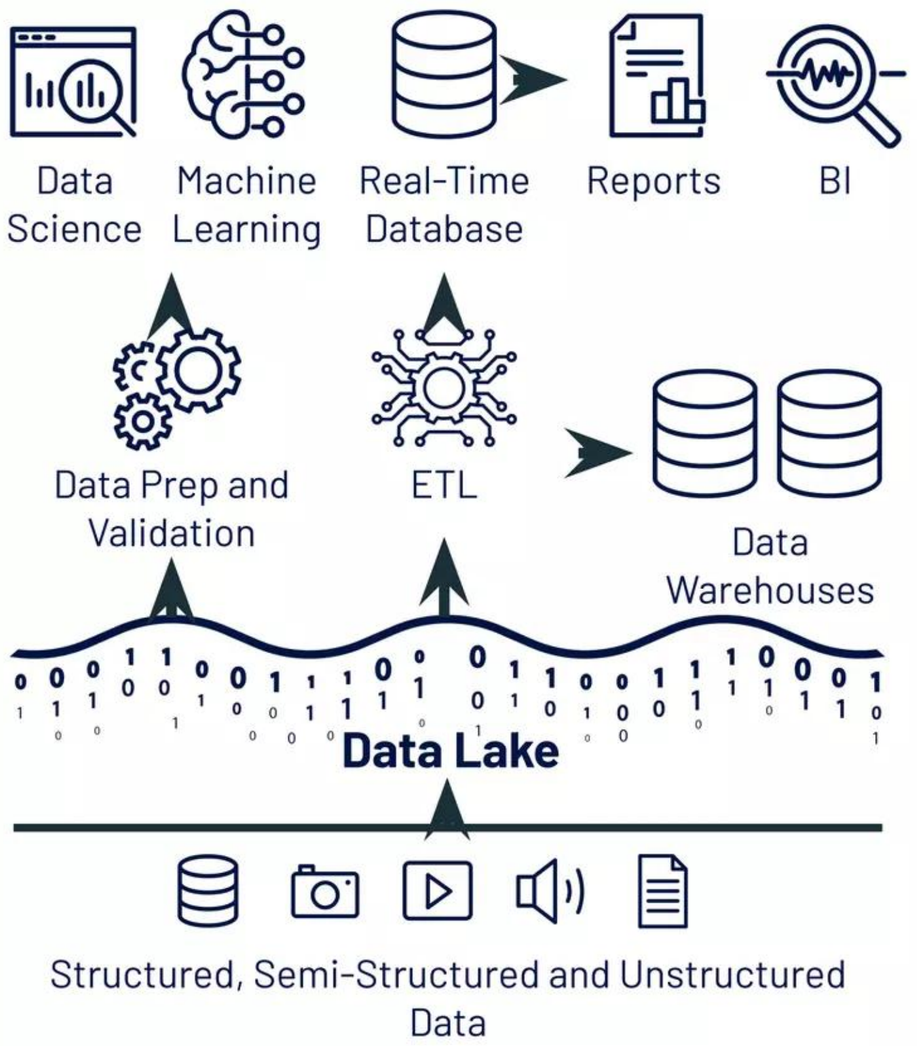
- No support for video, audio, text
- No support for data science, ML
- Limited support for streaming
- Closed & proprietary formats

Therefore, most data is stored in
data lakes & blob stores

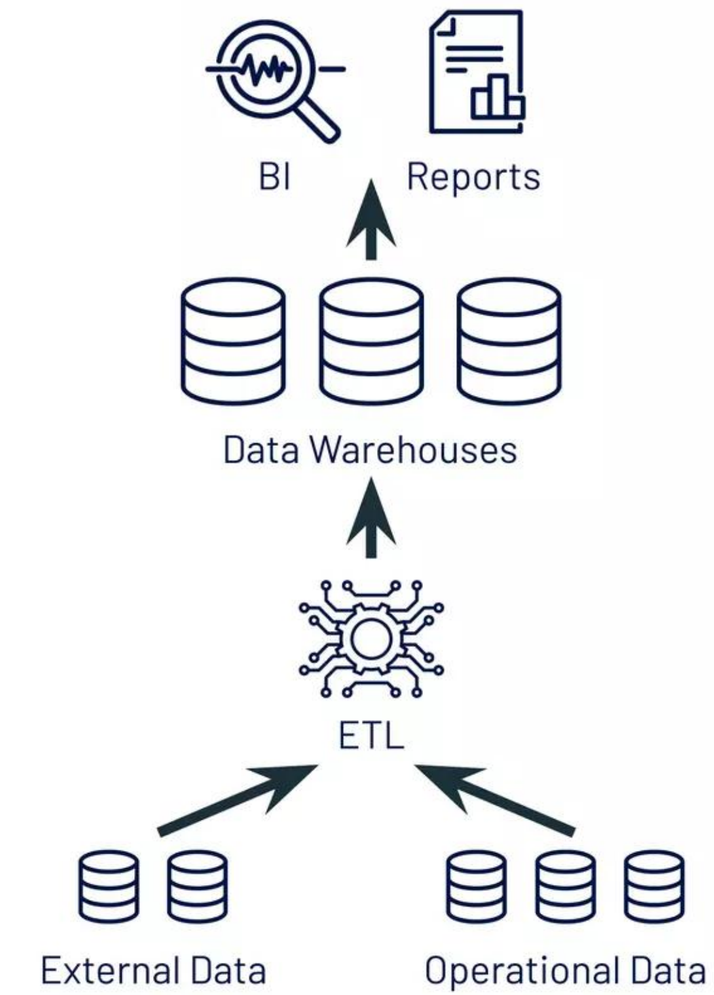


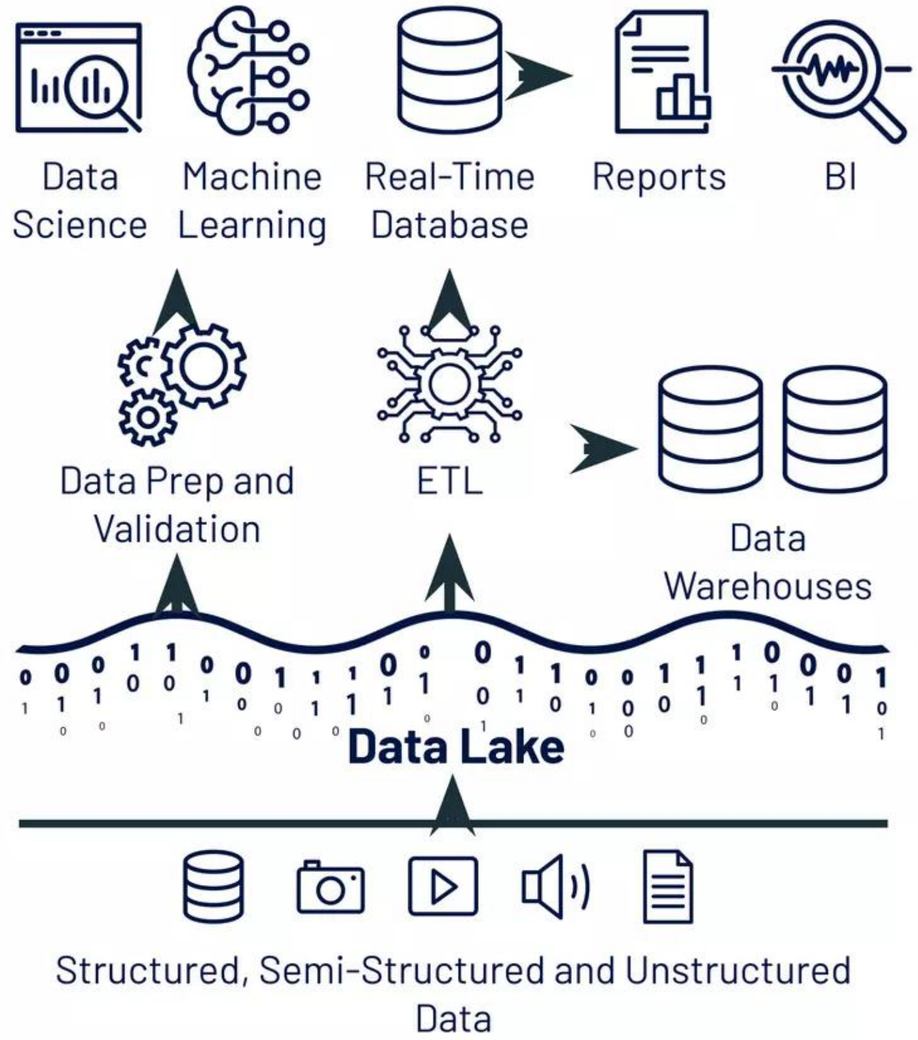
Data Lakes
could store **all** your data and determine what you want to know later

- Poor BI support
- Complex to set up
- Poor performance
- Unreliable data swamps



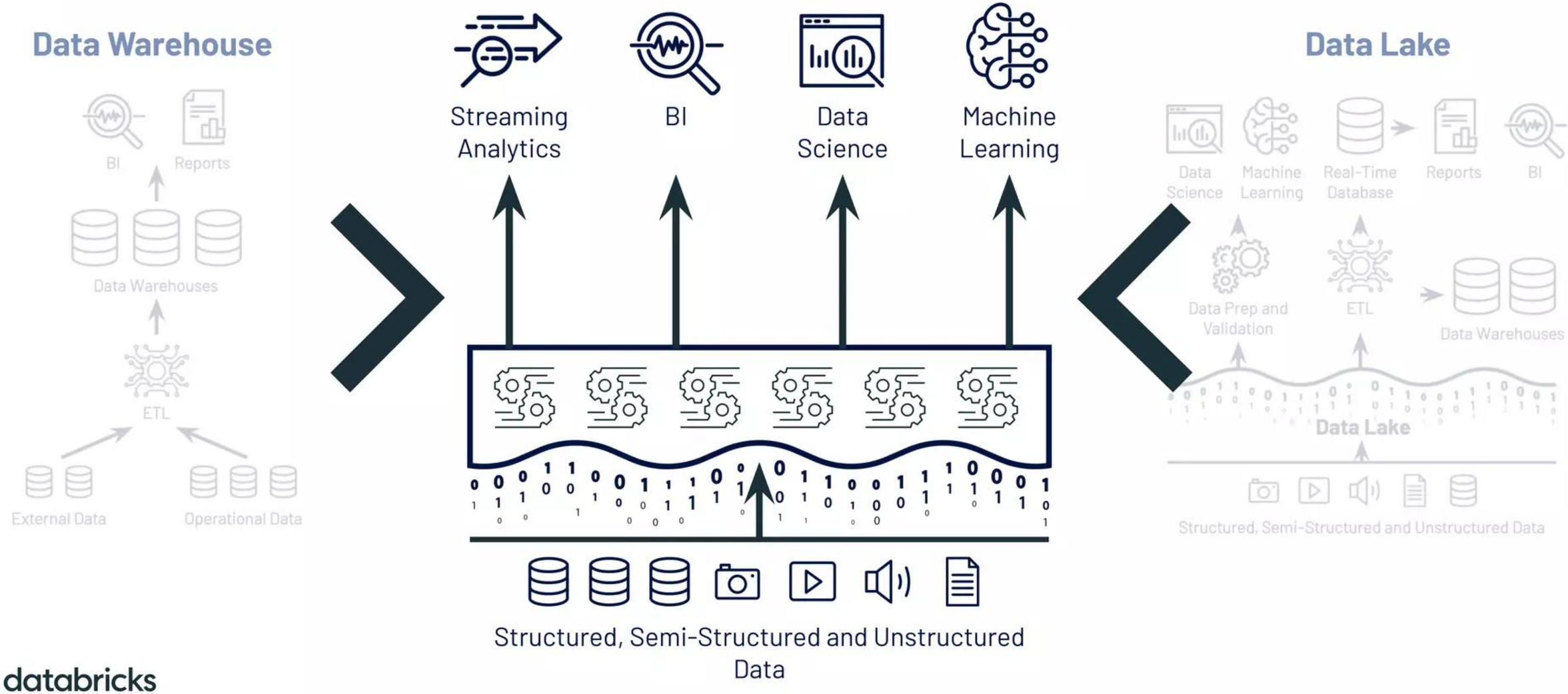
How do we get the best of both worlds?





databricks

Lakehouse



databricks

Lakehouse Summary

A Lakehouse has the following key features:

- support for diverse data types and formats
- data reliability and consistency
- support for diverse workloads (BI, data science, machine learning, and analytics)
- ability to use BI tools directly on source data

Building a Lakehouse

The core components we need to build a Lakehouse

1. Your data lake (cloud blob storage, open source format)
2. Transaction layer to provide consistency (Delta)
3. ETL and data cleansing workflow (Spark + Databricks Delta Pipelines)
4. Security, data integrity, and performance (Databricks Delta Engine)
5. As well as integrations for all of your user communities:
    a. SQL (Databricks SQL Analytics)
    b. BI tools and dashboards
    c. ML
    d. Streaming

Delta Lake



databricks

The Emergence of Data Lakes



Really cheap, durable storage
10 nines of durability. Cheap. Infinite scale.



Store all types of raw data
Video, audio, text, structured, unstructured

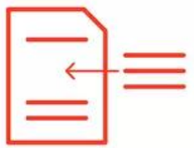


Open, standardized formats
Parquet format, big ecosystem of tools operate on these file formats

Challenges with data lakes



1. Hard to append data
Adding newly arrived data leads to incorrect reads



2. Modification of existing data is difficult
GDPR/CCPA requires making fine grained changes to existing data lake



3. Jobs failing mid way
Half of the data appears in the data lake, the rest is missing

Challenges with data lakes



4. Real-time operations
Mixing streaming and batch leads to inconsistency



5. Costly to keep historical versions of the data
Regulated environments require reproducibility, auditing, governance

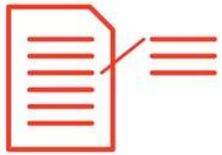


6. Difficult to handle large metadata
For large data lakes the metadata itself becomes difficult to manage

Challenges with data lakes



7. “Too many files” problems
Data lakes are not great at handling millions of small files



8. Hard to get great performance
Partitioning the data for performance is error-prone and difficult to change



9. Data quality issues
It’s a constant headache to ensure that all the data is correct and high quality

A new standard for building data lakes

An opinionated approach to building Data Lakes

- Adds reliability, quality, performance to Data Lakes
- Brings the best of data warehousing and data lakes
- Based on open source and open format (Parquet) - Delta Lake is also open source



1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. “Too many files” problems
8. Poor performance
9. Data quality issues

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. “Too many files” problems
8. Poor performance
9. Data quality issues

ACID Transactions

Make every operation transactional

- It either fully succeeds - or it is fully aborted for later retries

```
/path/to/table/_delta_log
 -  0000.json
 -  0001.json
 -  0002.json
 -  ...
 -  0010.parquet
```

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. "Too many files" problems
8. Poor performance
9. Data quality issues

ACID Transactions

Make every operation transactional

- It either fully succeeds - or it is fully aborted for later retries

```
/path/to/table/_delta_log
 -  0000.json   Add   file1.parquet
 -  0001.json   Add   file2.parquet
 -  0002.json   ...
 -  ...
 -  0010.parquet
```

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. "Too many files" problems
8. Poor performance
9. Data quality issues

ACID Transactions

Make every operation transactional

- It either fully succeeds - or it is fully aborted for later retries

```
/path/to/table/_delta_log
 -  0000.json
 -  0001.json   Remove  file1.parquet
 -  0002.json   Add     file3.parquet
 -  ...         ...
 -  0010.parquet
```

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. “Too many files” problems
8. Poor performance
9. Data quality issues

ACID Transactions

Make every operation transactional

- It either fully succeeds - or it is fully aborted for later retries

```
/path/to/table/_delta_log
 -  0000.json
 -  0001.json
 -  0002.json
 -  ...
 -  0010.parquet
 -  0010.json
 -  0011.json
```



1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. "Too many files" problems
8. Poor performance
9. Data quality issues

ACID Transactions

Make every operation transactional

- It either fully succeeds - or it is fully aborted for later retries

Review past transactions

- All transactions are recorded and you can go back in time to review previous versions of the data (i.e. *time travel*)

```
SELECT * FROM events
TIMESTAMP AS OF ...

SELECT * FROM events
VERSION AS OF ...
```

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. “Too many files” problems
8. Poor performance
9. Data quality issues

Spark under the hood

- Spark is built for handling large amounts of data
- All Delta Lake metadata stored in open Parquet format
- Portions of it cached and optimized for fast access
- Data and it’s metadata always co-exist. No need to keep catalog<>data in sync

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. "Too many files" problems
8. Poor performance
9. Data quality issues

File Consolidation

Automatically optimize a layout that enables fast access

- Partitioning: layout for typical queries
- Data skipping: prune files based on statistics on numericals
- Z-ordering: layout to optimize multiple columns

```
OPTIMIZE events
ZORDER BY (eventType)
```

1. Hard to append data
2. Modification of existing data difficult
3. Jobs failing mid way
4. Real-time operations hard
5. Costly to keep historical data versions
6. Difficult to handle large metadata
7. “Too many files” problems
8. Poor performance
9. Data quality issues

Schema validation

Schema validation and evolution

- All data in Delta Tables have to adhere to a strict schema (star, etc)
- Includes schema evolution in merge operations

```
MERGE INTO events
USING changes
ON events.id = changes.id
WHEN MATCHED THEN
  UPDATE SET *
WHEN NOT MATCHED THEN
  INSERT *
```

Delta Lake Summary

- Core component of a Lakehouse architecture
- Offers guaranteed consistency because it's ACID compliant
- Robust data store
- Designed to work with Apache Spark

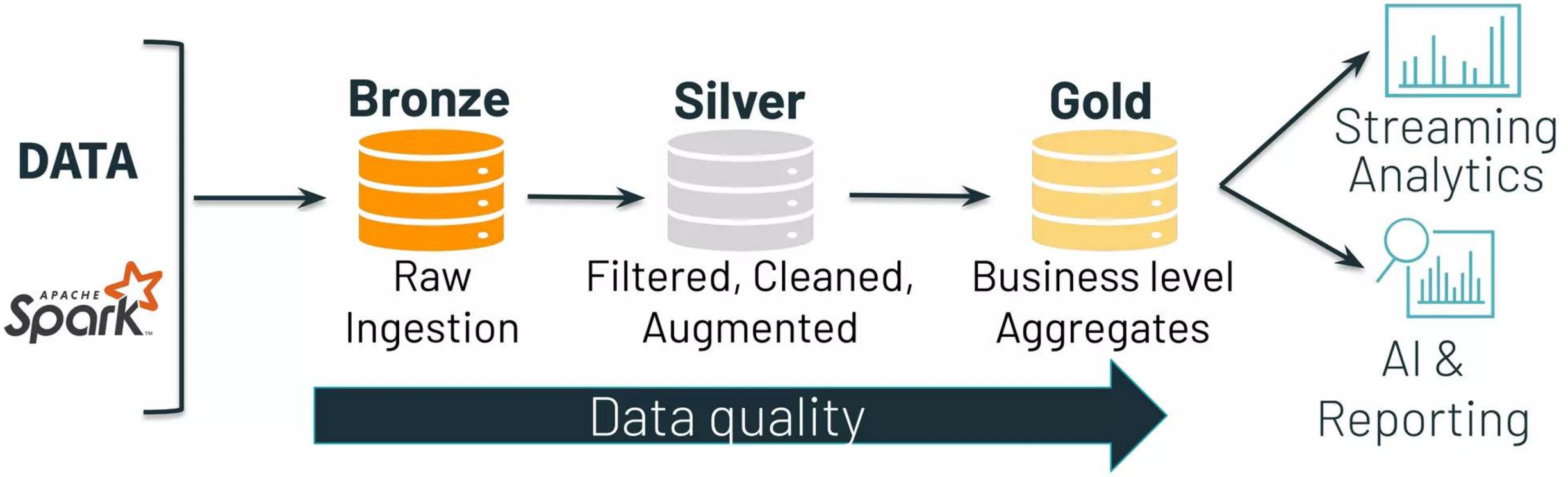


Elements of Delta Lake

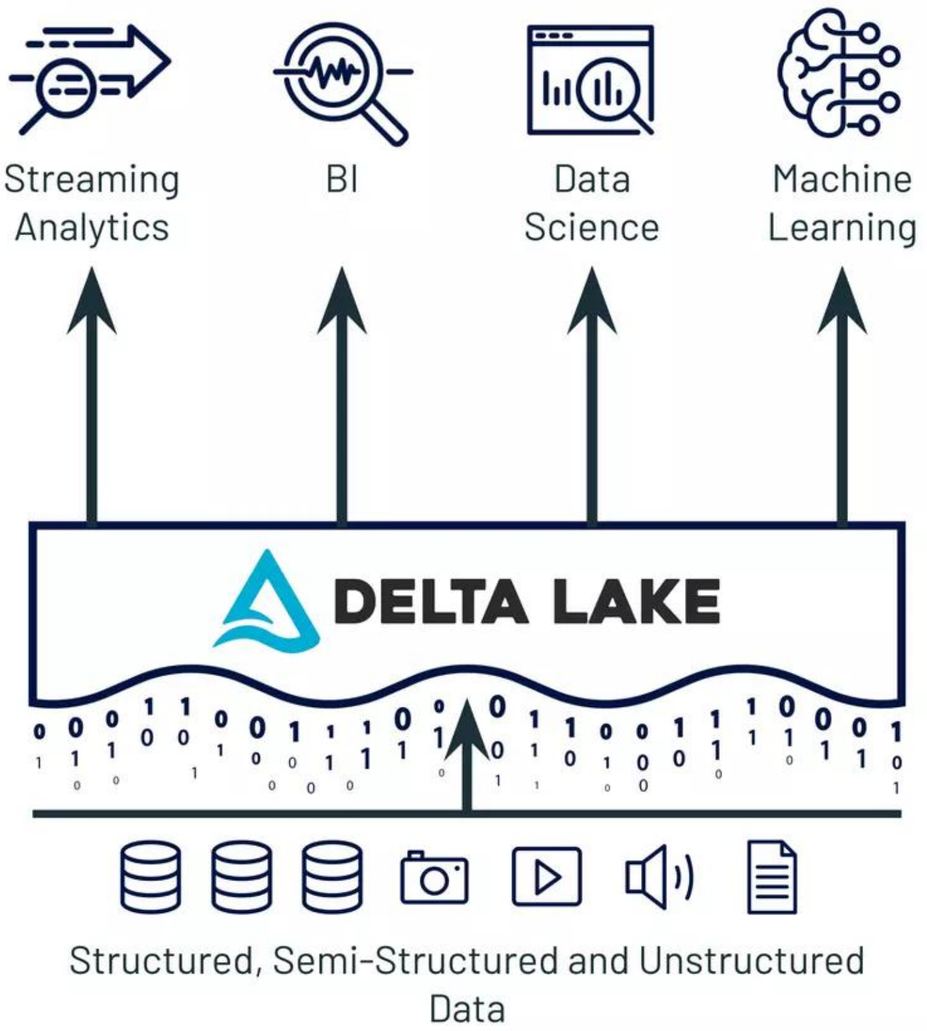


- Delta Architecture
- Delta Storage Layer
- Delta Engine

Delta architecture



Delta Storage Layer



One platform for every use case

Structured transactional layer

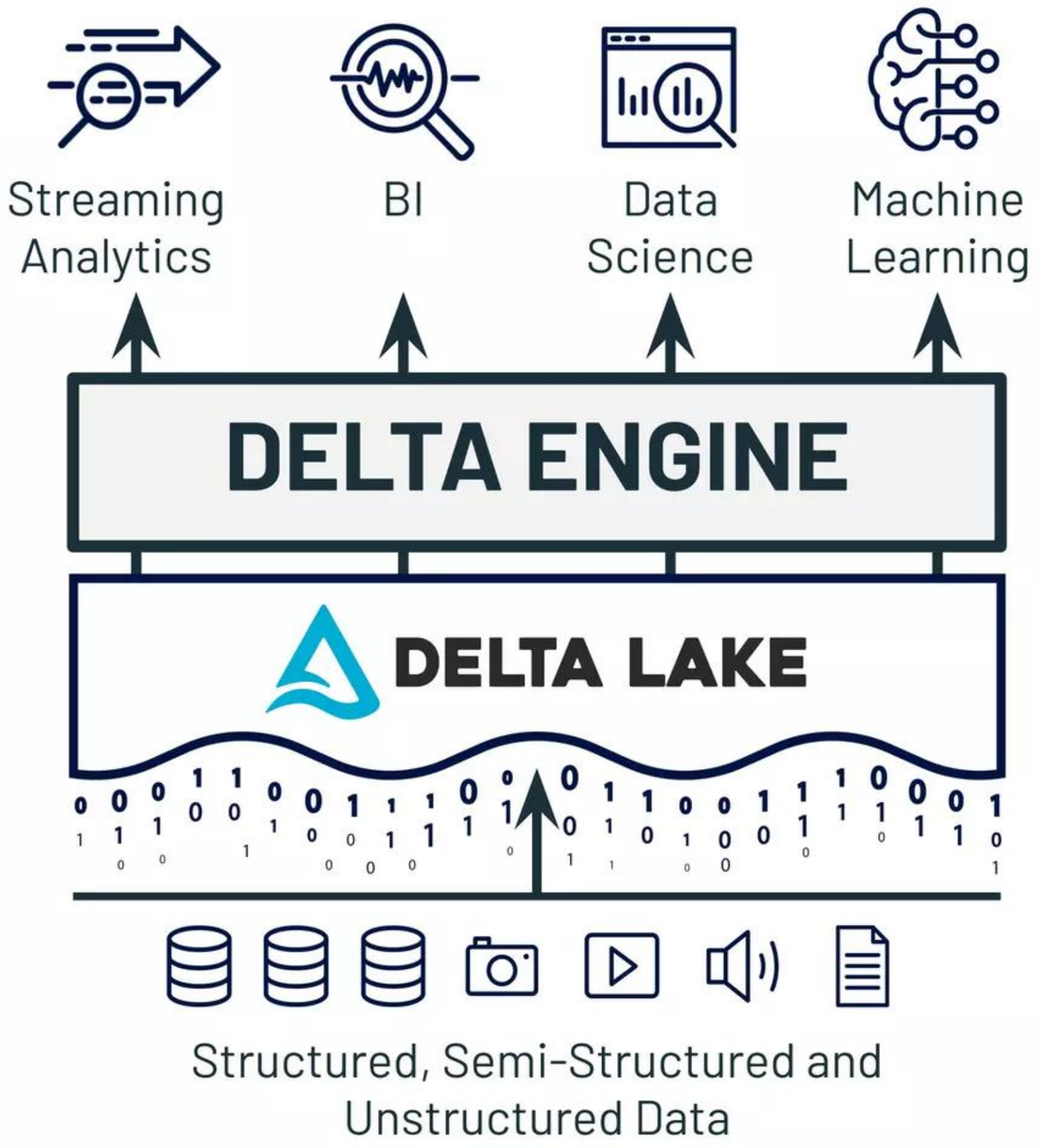
Data Lake for all your data

databricks

Databricks' Delta Engine

Performance

- File management optimizations
- Performance optimization with Delta Caching
- Dynamic File Pruning
- Adaptive Query Execution





One platform for every use case

High performance query engine

Structured transactional layer

Data Lake for all your data

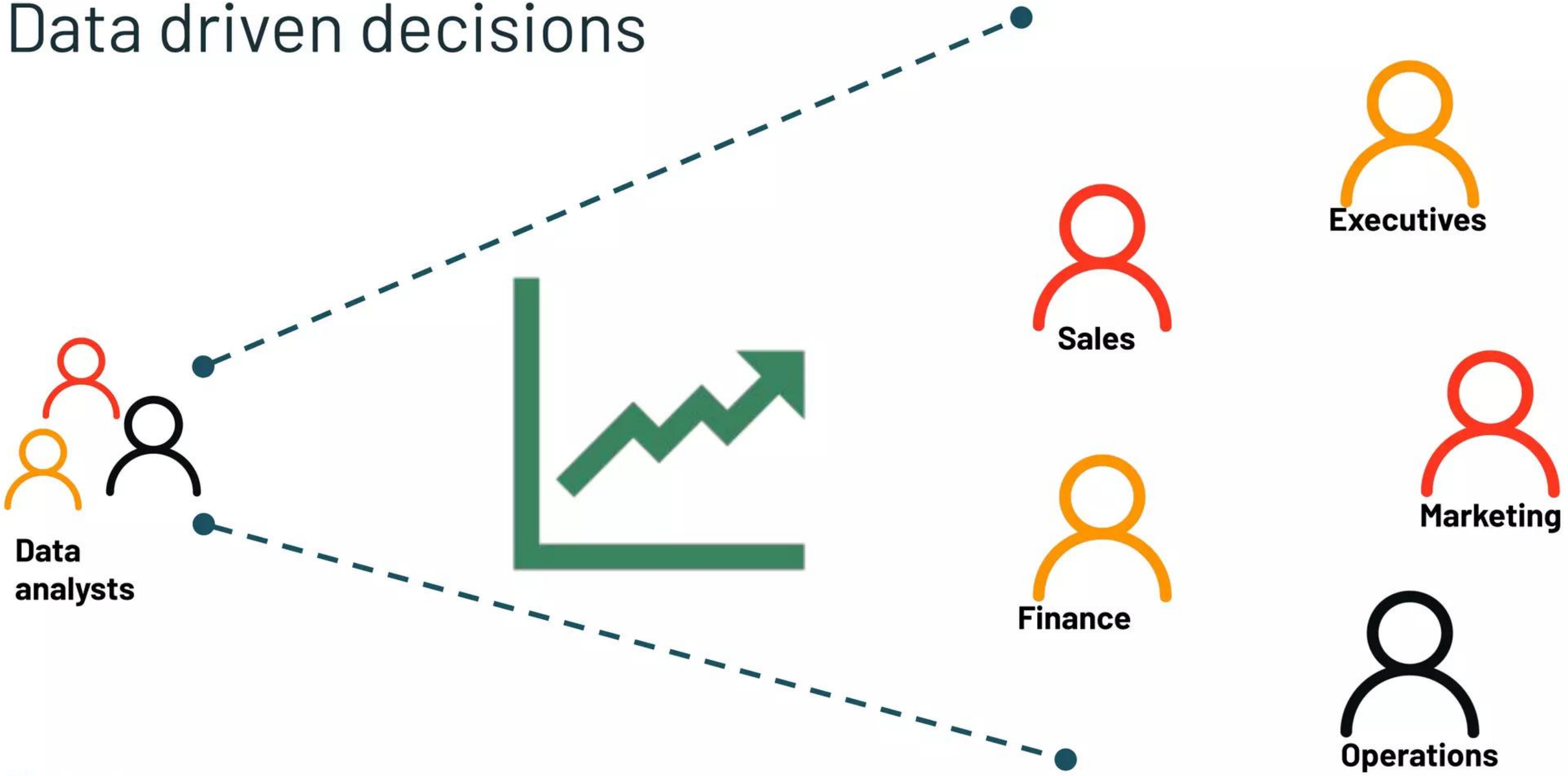
databricks

SQL Analytics



databricks

Data driven decisions



Challenges solved by Delta Lake

Incomplete data silos



Stale data

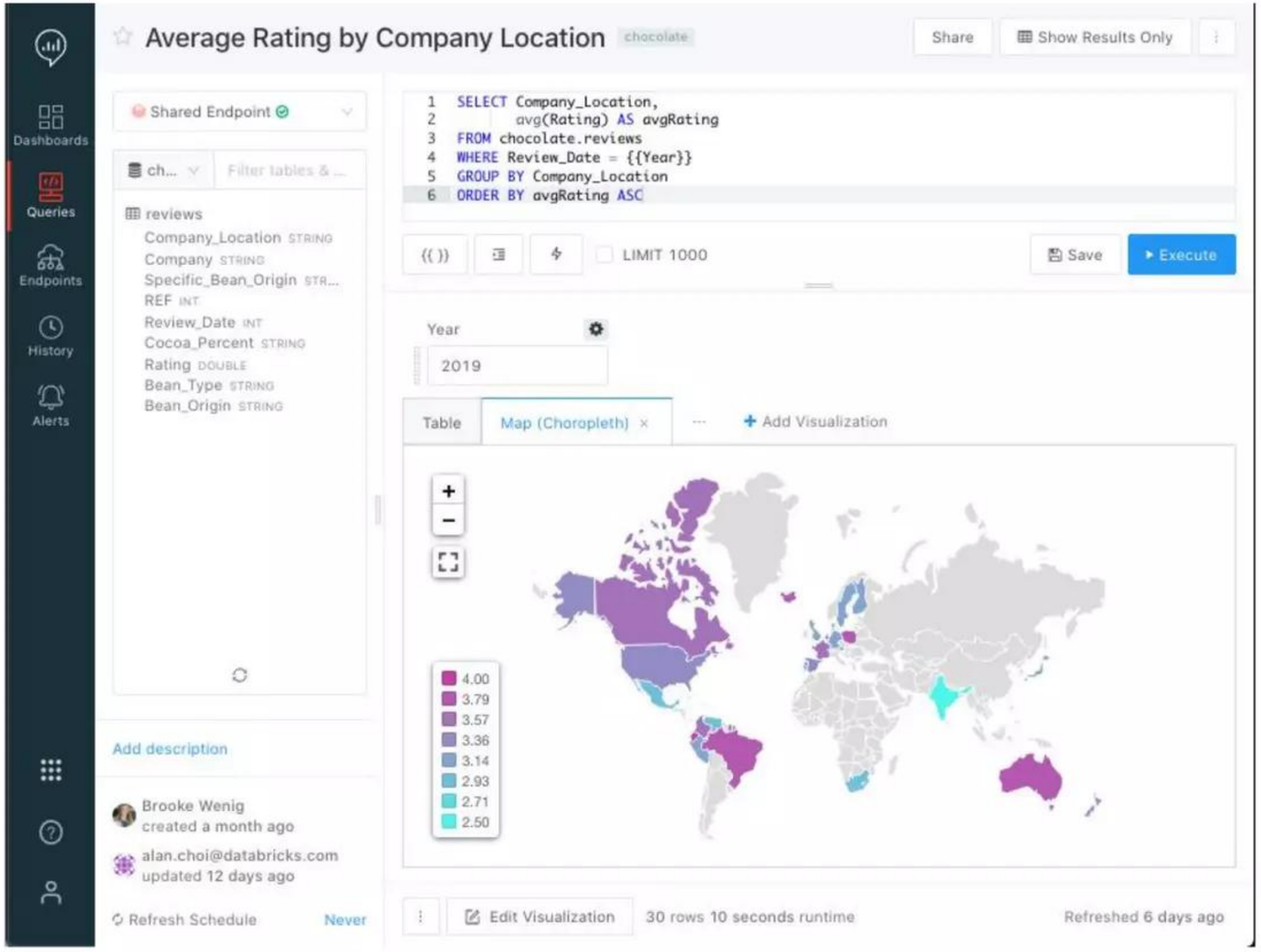


Complexity



SQL-native user interface

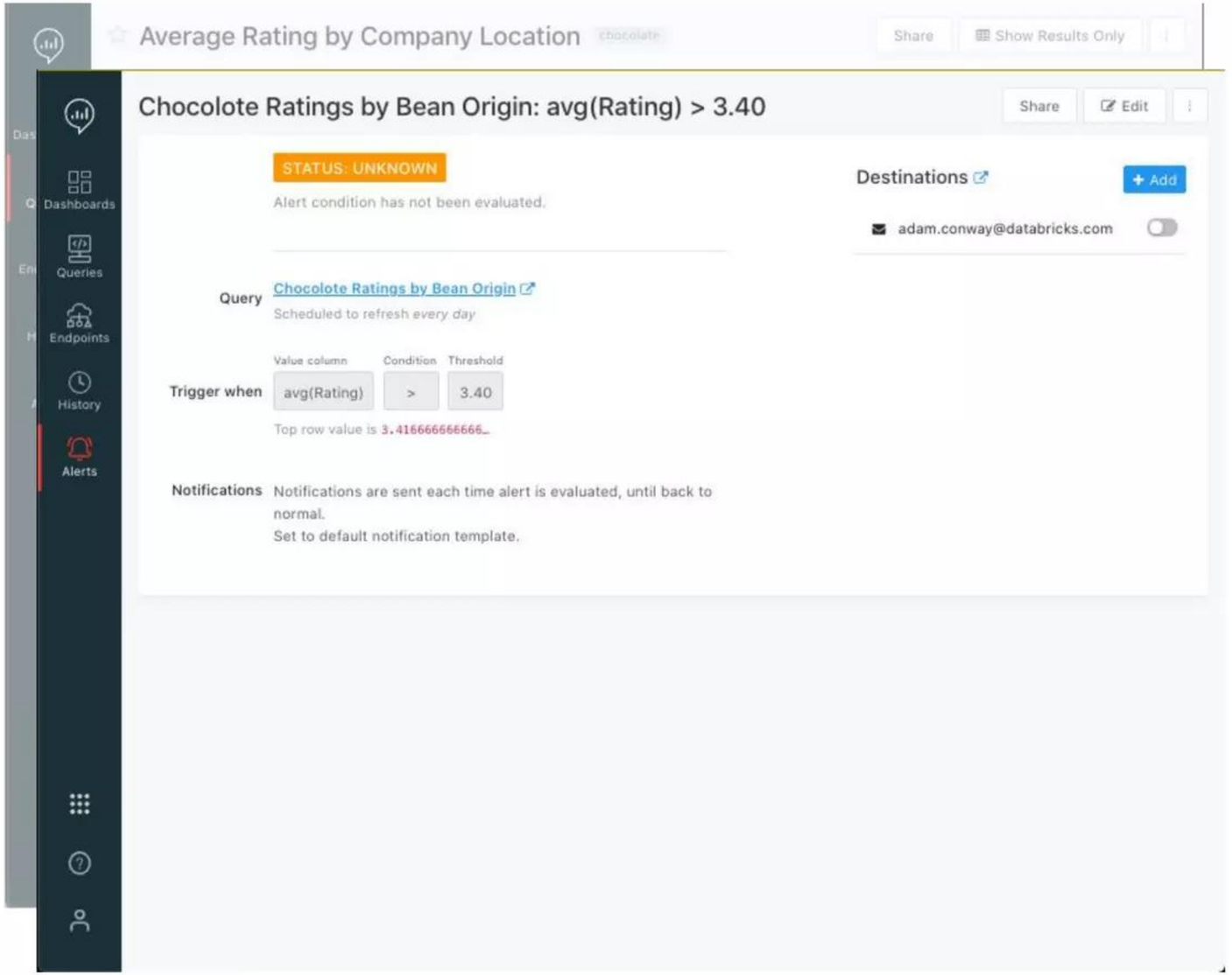
- Familiar SQL Editor
  - Auto Complete
  - Built in visualizations
  - Data Browser



databricks

SQL-native user interface

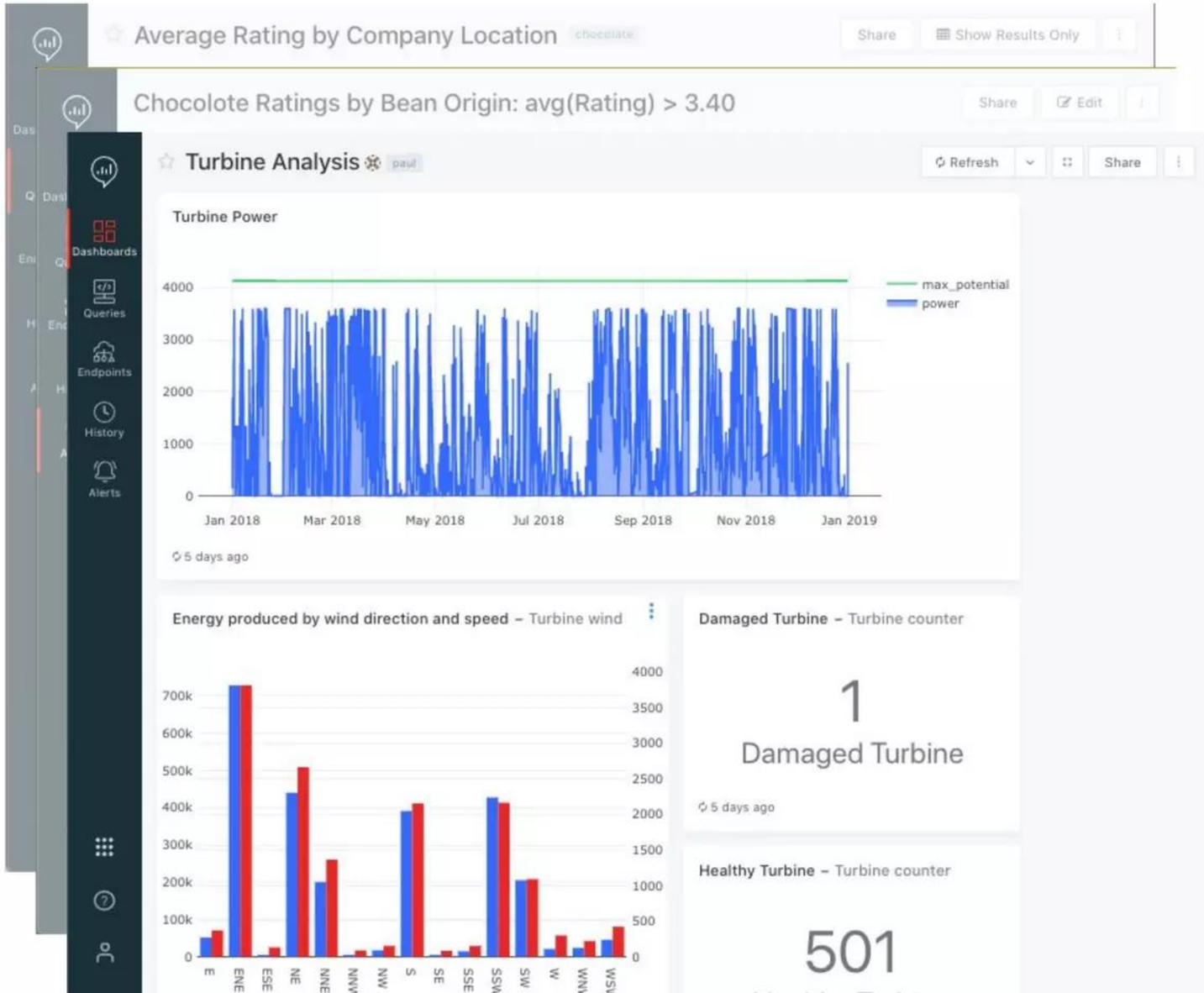
- Familiar SQL Editor
  - Auto Complete
  - Built in visualizations
  - Data Browser
- Automatic Alerts
  - Trigger based upon values
  - Email or Slack integration



databricks

SQL-native user interface

- Familiar SQL Editor
  - Auto Complete
  - Built in visualizations
  - Data Browser
- Automatic Alerts
  - Trigger based upon values
  - Email or Slack integration
- Dashboards
  - Simply convert queries to dashboards
  - Share with Access



databricks

Built-in connectors for existing BI tools











ThoughtSpot

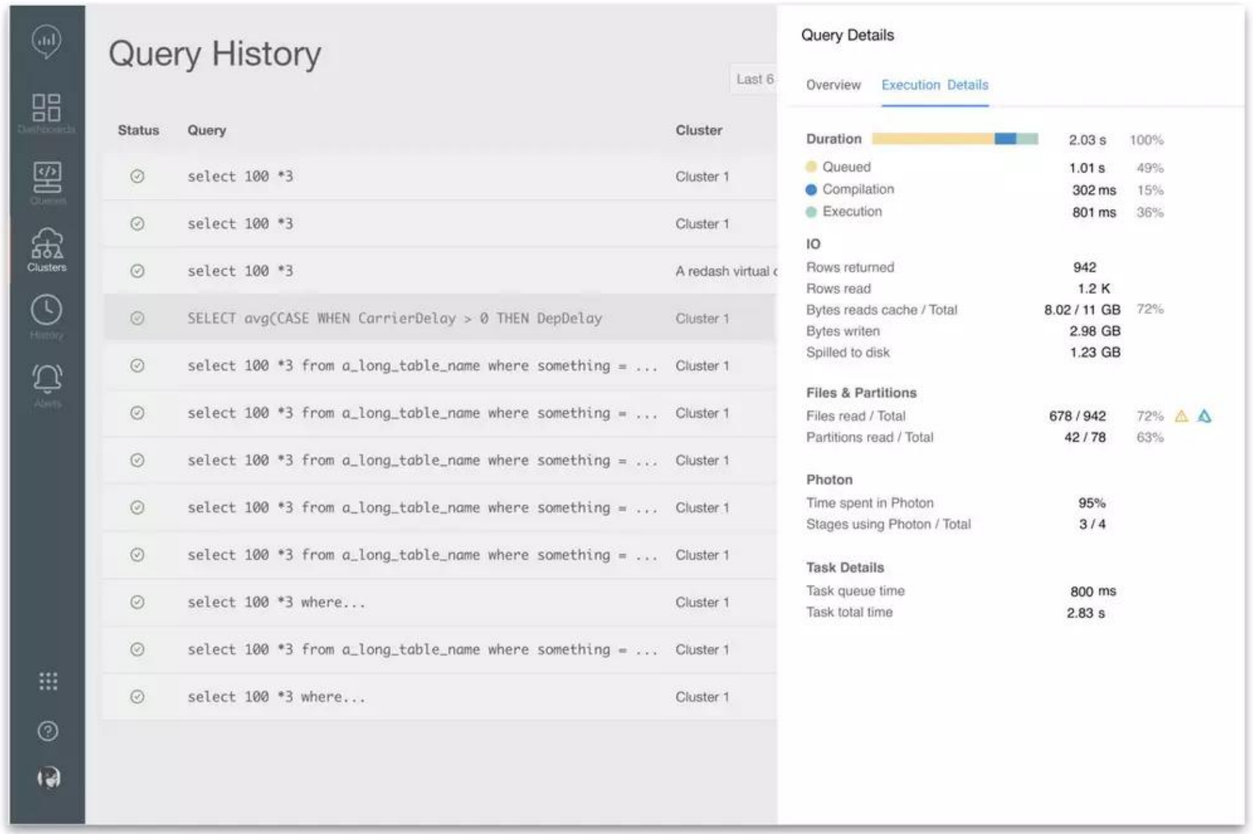
Other BI & SQL clients that support



- Supports your favorite tool
  - Connectors for top BI & SQL clients
  - Simple connection setup
  - Optimized performance
- OAuth & Single Sign On
  - Quick and easy authentication experience. No need to deal with access tokens.
  - Power BI Available now
  - Others coming soon



SQL Analytics Demo



databricks

Join us for Part 2

Login and use SQL Analytics hands-on:

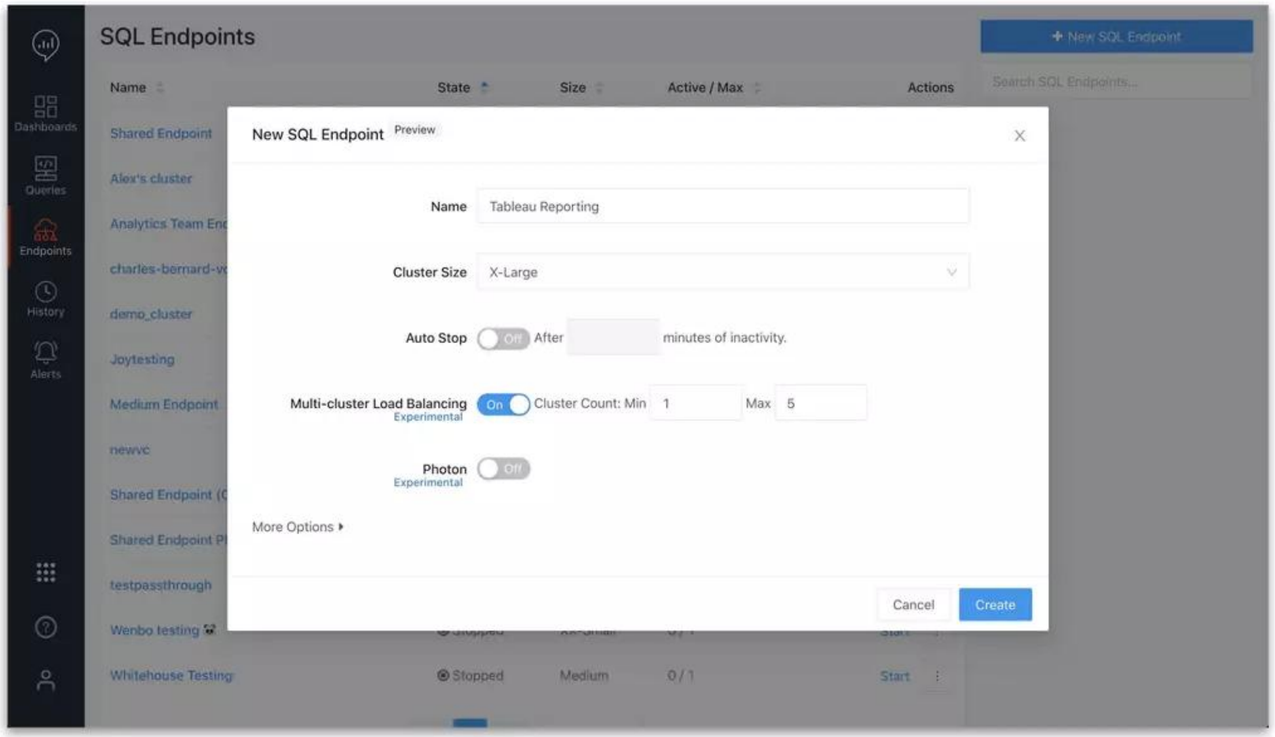
Dec 15 at 10am (San Francisco Time)

Thanks for coming!



Setup & Administration

SQL Endpoints



SQL Optimized Compute

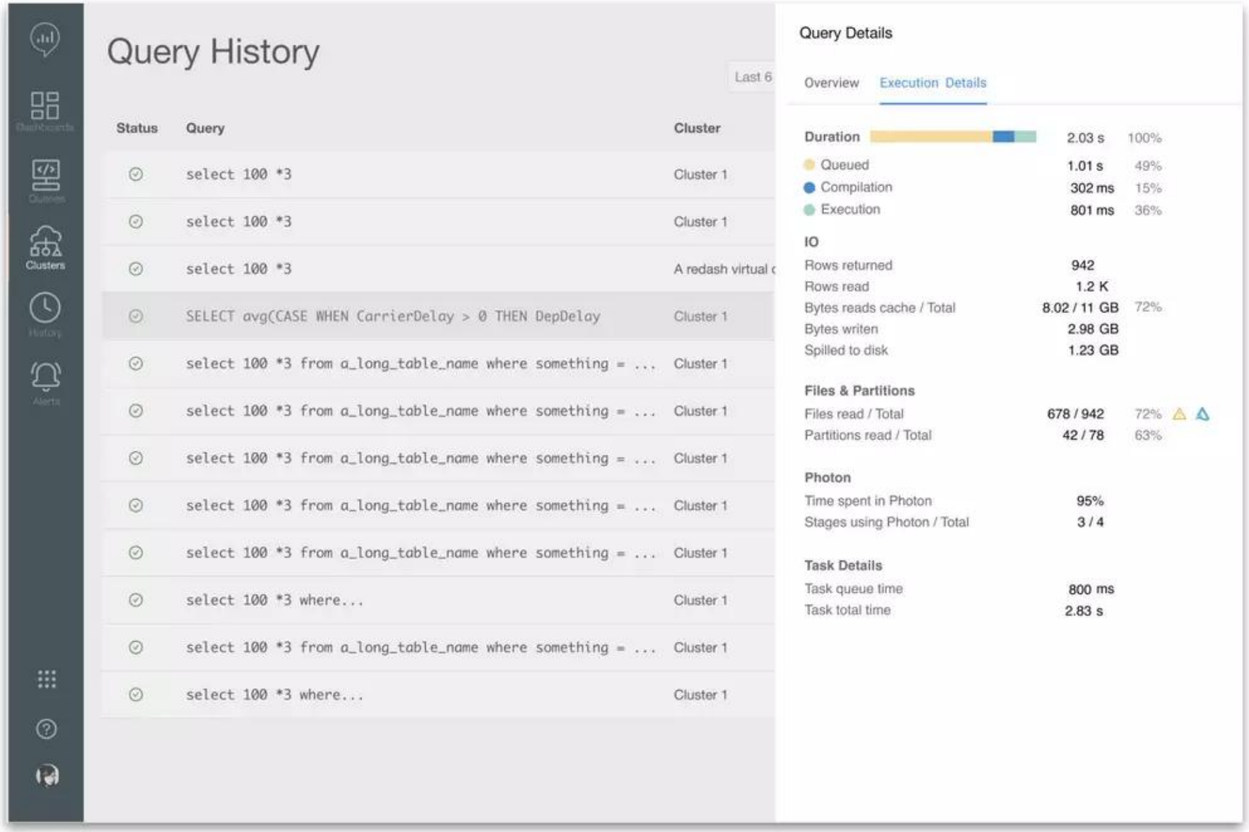
SQL Endpoints give a quick way to setup SQL / BI optimized compute. You pick a tshirt size. Databricks will ensure configuration that provides the highest price/performance.

Concurrency Scaling Built-in

[Private Preview]

Virtual clusters can load balance queries across multiple clusters behind the scenes, providing unlimited concurrency.

Query History

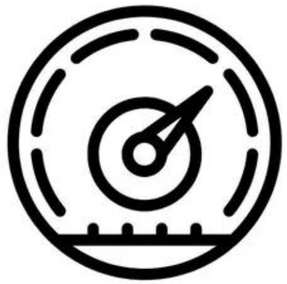


Central Query Log

Track & understand usage across virtual clusters, users & time. Easily observe workloads across Redash, BI tools & any other SQL client usage.

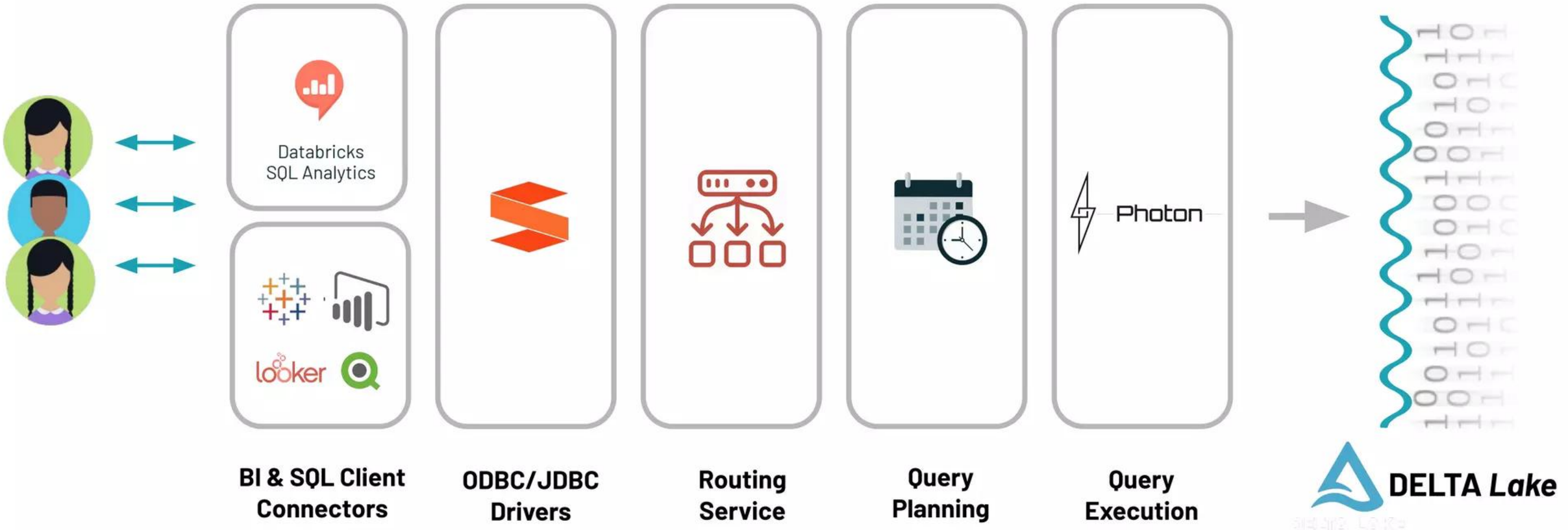
Troubleshoot & debug

History is the starting point for understanding / triaging any errors & performance issues. Jump into detailed Spark query profile as needed.



Performance

Performance – Life of a Query



Up to 9x better price/performance

